6.2 컨센서스 메커니즘 분석

개요

Composable Finance 프로젝트는 파라체인으로서 Polkadot/Kusama 릴레이체인의 공유 보안을 활용하면서, 자체적으로는 AURA (Authority Round) 컨센서스를 사용하여 블록을 생성합니다. 이 문서에서는 AURA 블록 생성 과정, 파라체인 컨센서스와 릴레이체인 컨센서스의 상호작용, 그리고 콜레이터 선택 메커니즘을 상세히 분석합니다.

6.2.1 AURA (Authority Round) 블록 생성 과정

AURA 컨센서스 기본 구성

AURA는 권한 기반 라운드 로빈 컨센서스 알고리즘으로, 사전에 정의된 권한자(Authority) 목록에 따라 순차적으로 블록을 생성합니다.

런타임 설정

Picasso와 Composable 런타임 모두 동일한 AURA 설정을 사용합니다:

```
// Picasso 런타임 (code/parachain/runtime/picasso/src/lib.rs)

parameter_types! {

// Maximum authorities/collators for aura

pub  const MaxAuthorities: u32 = 100;

}


impl aura::Config for Runtime {
```

```
type AuthorityId = AuraId;

type DisabledValidators = ();

type MaxAuthorities = MaxAuthorities;

}



impl cumulus_pallet_aura_ext::Config for Runtime {}
```

세션 키 구성

AURA는 세션 시스템과 통합되어 권한자 관리를 수행합니다:

```
impl_opaque_keys! {

pub  struct SessionKeys {

pub  aura: Aura,

}

}
```

슬롯 기반 블록 생성

AURA는 고정된 시간 간격(슬롯)에 따라 블록을 생성합니다:

```
// 타임스탬프 설정에서 AURA와 연동

impl timestamp::Config for Runtime {

type Moment = Moment;

/// What to do when SLOT_DURATION has passed?

type OnTimestampSet = Aura;
```

```
type MinimumPeriod = MinimumPeriod;

type WeightInfo = weights::timestamp::WeightInfo<Runtime>;

}
```

슬롯 지속 시간은 `SLOT_DURATION` 상수로 정의되며, 일반적으로 12초로 설정됩니다.

블록 생성 과정

1. **슬롯 계산**: 현재 시간을 기준으로 슬롯 번호를 계산
2. **권한자 선택**: 슬롯 번호를 권한자 수로 나눈 나머지로 현재 슬롯의 권한자 결정
3. **블록 제안**: 선택된 권한자가 블록을 제안하고 서명
4. **블록 전파**: 생성된 블록을 네트워크에 전파

6.2.2 파라체인 컨센서스 구현

Cumulus AURA 확장

파라체인은 `cumulus_pallet_aura_ext` 를 사용하여 AURA를 파라체인 환경에 맞게 확장합니다:

```
impl cumulus_pallet_aura_ext::Config for Runtime {}
```

이 확장은 다음과 같은 기능을 제공합니다:

- **릴레이체인 동기화**: 릴레이체인 블록과 파라체인 블록의 동기화
- **검증 데이터 처리**: 릴레이체인으로부터 받은 검증 데이터 처리
- **블록 실행**: 파라체인 블록의 실행 및 검증

파라체인 블록 실행자

파라체인은 특별한 블록 실행자를 사용합니다:

```
cumulus_pallet_parachain_system::register_validate_block!(
Runtime = Runtime,
BlockExecutor = cumulus_pallet_aura_ext::BlockExecutor::<Runtime, Executive>,
CheckInherents = CheckInherents,
);
```

이 실행자는 다음을 담당합니다:

- **고유 데이터 검증**: 파라체인 고유 데이터(inherent data) 검증
- **상태 전환**: 블록 실행을 통한 상태 전환
- **증명 생성**: 릴레이체인 검증을 위한 상태 증명 생성

AURA API 구현

런타임은 AURA 관련 API를 제공합니다:

```
impl sp_consensus_aura::AuraApi<Block, AuraId> for Runtime {
fn slot_duration() -> sp_consensus_aura::SlotDuration {
sp_consensus_aura::SlotDuration::from_millis(Aura::slot_duration())
}

fn authorities() -> Vec<AuraId> {
Aura::authorities().into_inner()
}
```

```
}
```

6.2.3 콜레이터 선택 및 세션 관리

콜레이터 선택 팰릿

콜레이터 선택은 `collator_selection` 팰릿을 통해 관리됩니다:

```
impl authorship::Config for Runtime {
type FindAuthor = session::FindAccountFromAuthorIndex<Self, Aura>;
type EventHandler = (CollatorSelection,);
}
```

세션 설정

세션 시스템은 콜레이터 로테이션을 관리합니다:

```
impl session::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type ValidatorId = <Self  as system::Config>::AccountId;
type ValidatorIdOf = ConvertInto;
type ShouldEndSession = session::PeriodicSessions<Period, Offset>;
type NextSessionRotation = session::PeriodicSessions<Period, Offset>;
type SessionManager = CollatorSelection;
// Essentially just Aura, but lets be pedantic.
type SessionHandler =
```

```
<opaque::SessionKeys as sp_runtime::traits::OpaqueKeys>::KeyTypeIdProviders;

type Keys = opaque::SessionKeys;

type WeightInfo = weights::session::WeightInfo<Runtime>;

}
```

주요 설정 요소:

- **세션 기간**: `Period` 로 정의된 블록 수만큼 세션 지속
- **세션 오프셋**: `Offset` 으로 정의된 블록에서 세션 시작
- **세션 관리자**: `CollatorSelection` 팰릿이 콜레이터 선택 관리

6.2.4 노드 레벨 컨센서스 구현

서비스 레벨 AURA 설정

노드 서비스에서 AURA 컨센서스를 초기화합니다:

```
// service.rs에서

let  slot_duration = cumulus_client_consensus_aura::slot_duration(&*client)?;



let  parachain_consensus =

AuraConsensus::build::<sp_consensus_aura::sr25519::AuthorityPair, _, _, _, _, _,

BuildAuraConsensusParams {

proposer_factory,

create_inherent_data_providers: move |_, (relay_parent, validation_data)| {

let  relay_chain_interface_inherent = relay_chain_interface_inherent.clone();
```

```
async  move {
let  parachain_inherent = cumulus_primitives_parachain_inherent::ParachainInhere
relay_parent,
&relay_chain_interface_inherent,
&validation_data,
para_id,
).await;
let  time = sp_timestamp::InherentDataProvider::from_system_time();
let  slot = sp_consensus_aura::inherents::InherentDataProvider::from_timestamp_a
*time,
slot_duration,
);
let  parachain_inherent = parachain_inherent.ok_or_else(|| {
Box::<dyn std::error::Error + Send + Sync>::from(
"Failed to create parachain inherent",
)
})?;
Ok((slot, time, parachain_inherent))
}
},
block_import: parachain_block_import,
para_client: client.clone(),
backoff_authoring_blocks,
```

```
sync_oracle: sync_service.clone(),

keystore,

force_authoring,

slot_duration,

telemetry: telemetry.as_ref().map(|t| t.handle()),

// We got around 500ms for proposing

block_proposal_slot_portion: SlotProportion::new(1_f32 / 24_f32),

// And a maximum of 750ms if slots are skipped

max_block_proposal_slot_portion: Some(SlotProportion::new(1_f32 / 16_f32)),

},

);
```

고유 데이터 제공자

AURA 컨센서스는 다음과 같은 고유 데이터를 처리합니다:

1. **타임스탬프**: 블록 생성 시간
2. **슬롯**: AURA 슬롯 정보
3. **파라체인 고유 데이터**: 릴레이체인으로부터의 검증 데이터

블록 제안 시간 제한

- **기본 제안 시간**: 슬롯의 1/24 (약 500ms)
- **최대 제안 시간**: 슬롯의 1/16 (약 750ms)

이러한 시간 제한은 파라체인이 릴레이체인의 타이밍 요구사항을 충족하도록 보장합니다.

6.2.5 파라체인과 릴레이체인 컨센서스 상호작용

릴레이체인 슬롯 동기화

파라체인은 릴레이체인의 슬롯과 동기화됩니다:

```
let  relay_chain_slot_duration = Duration::from_secs(6);
```

릴레이체인의 슬롯 지속 시간은 6초이며, 파라체인은 이에 맞춰 블록을 생성합니다.

검증 데이터 처리

파라체인 블록은 릴레이체인으로부터 검증 데이터를 받아 처리합니다:

```
let  parachain_inherent = cumulus_primitives_parachain_inherent::ParachainInhere
relay_parent,
&relay_chain_interface_inherent,
&validation_data,
para_id,
).await;
```

이 데이터에는 다음이 포함됩니다:

- **릴레이 부모 해시**: 릴레이체인 부모 블록 해시
- **검증 데이터**: 파라체인 검증을 위한 데이터
- **파라체인 ID**: 현재 파라체인의 식별자

최종성 보장

파라체인은 자체적으로 최종성을 제공하지 않고, 릴레이체인의 GRANDPA 최종성에 의존합니다:

1. **파라체인 블록 생성**: AURA를 통한 블록 생성
2. **릴레이체인 포함**: 생성된 블록이 릴레이체인에 포함
3. **GRANDPA 최종화**: 릴레이체인 블록이 GRANDPA에 의해 최종화
4. **파라체인 최종화**: 릴레이체인 최종화와 함께 파라체인 블록도 최종화

6.2.6 백오프 및 오류 처리

블록 생성 백오프

노드는 최종화 지연 시 블록 생성을 백오프합니다:

```
let  backoff_authoring_blocks =
Some(sc_consensus_slots::BackoffAuthoringOnFinalizedHeadLagging::default());
```

이는 네트워크 지연이나 동기화 문제 시 불필요한 블록 생성을 방지합니다.

강제 블록 생성

개발 환경에서는 강제 블록 생성을 활성화할 수 있습니다:

```
force_authoring, // 설정에서 가져온 값
```

이는 테스트넷이나 개발 환경에서 지속적인 블록 생성을 보장합니다.

결론

Composable Finance의 컨센서스 메커니즘은 AURA를 기반으로 한 효율적인 파라체인 블록 생성 시스템입니다. 릴레이체인과의 긴밀한 동기화를 통해 보안성을 보장하면서도, 콜레이터 선택과 세션 관

리를 통해 탈중앙화를 유지합니다. 이러한 설계는 높은 처리량과 낮은 지연시간을 제공하면서도 Polkadot 생태계의 공유 보안 혜택을 누릴 수 있게 해줍니다.